1 Cross-Compilation Toolchain

1.1 Host, Target, and Emulator

Role	Meaning
Host	Computer running the editor, `make`, and the toolchain (x86-64 or Apple silicon)
Target	ARM Cortex-M3 machine for which the ELF is built
Emulator	QEMU model that executes target instructions on the host

A native `gcc` emits code for the machine it runs on; it cannot produce Cortex-M3 firmware just because the source language is C. A **cross toolchain** (`arm-none-eabi-*`) runs on the host but emits code for the target instruction set architecture (ISA).

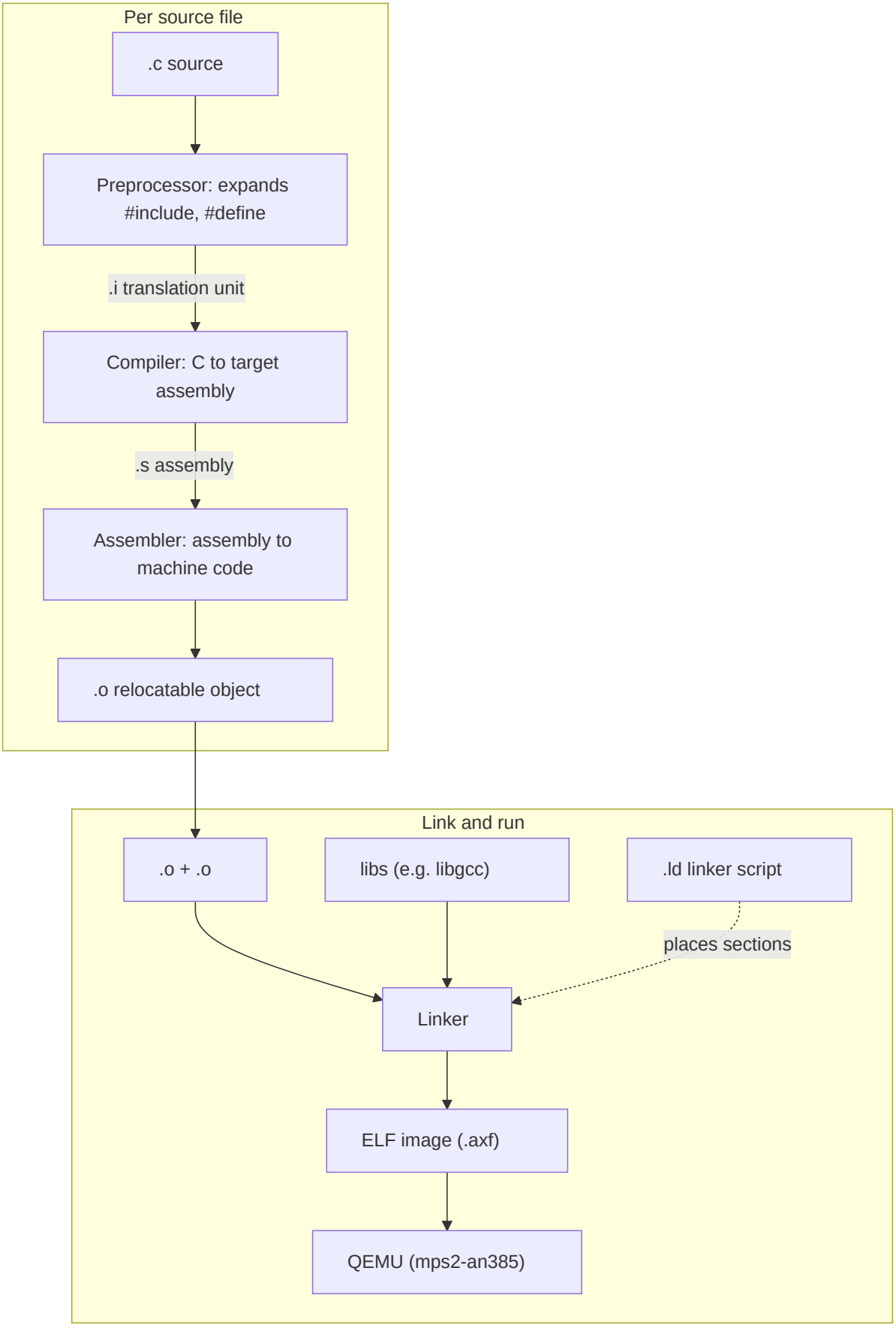
```
gcc -dumpmachine              # host triple, e.g. x86_64-linux-gnu
arm-none-eabi-gcc -dumpmachine  # target triple: arm-none-eabi
```

The triple `arm-none-eabi` means: ARM architecture, **no** operating system on the target, standard ARM **E**mbedded **ABI**.

- Consequences of "no OS":
  - No `_start` from a C library unless the application provides one.
  - No dynamic loader, and no process to return to when `main()` finishes.
  - No guaranteed `printf` console, no `malloc` backed by an OS.
- QEMU is not the target CPU, but it provides a reproducible board model before physical hardware is used.

1.2 Build Pipeline Stages

Figure 2.1 — Build pipeline from source files to the ELF image



Stage	What it does	Stop with	Output
Preprocessing	Expands `#include`, macros, conditional compilation; pure text substitution	`-E`	`.i` translation unit
Compilation	Checks syntax and types, **optimises**, emits **ARM assembly** for the selected CPU and ABI	`-S`	`.s`
Assembly	Encodes assembly into a **relocatable object file**	`-c`	`.o`
Linking	Combines objects, resolves symbols, assigns final addresses	(full run)	ELF image

- `arm-none-eabi-gcc` is a driver that invokes each stage in turn; `make` decides which files need rebuilding.
- **Preprocessing** produces one **translation unit**: a single file of plain C with every include and macro expanded.

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding \
    -E src/main.c -o build/main.i
```

In `main.i`, `#include <stdint.h>` is replaced inline by its typedefs; `main()` is unchanged text with comments stripped. `-E` only substitutes text and does not touch program logic.

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding \
    -S src/main.c -o build/main.s
```

In `main.s`, the `bkpt 0xab` instruction from the inline assembly in `main.c` passes through untouched; everything around it is Thumb-2 assembly generated from the C source.

1.3 Target Compiler and Linker Flags

```
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -std=gnu17
CFLAGS += -Wall -Wextra -O0 -g3

LDFLAGS := -T scripts/mps2_m3.ld
LDFLAGS += -Xlinker -Map=build/output.map
```

Flag	Meaning
`-mthumb -mcpu=cortex-m3`	Selects the Thumb instruction subset implemented by the Cortex-M3
`-nostartfiles`	Excludes the toolchain's default C startup objects, so reset handling is done by the project's own startup code
`-ffreestanding`	Compiles for a freestanding (bare-metal) C environment, without hosted-library assumptions
`-std=gnu17`	C17 with GNU extensions

Flag	Meaning
`-Wall -Wextra`	Warning policy
`-O0 -g3`	No optimisation and full debug information: a debuggable, source-corresponding image
`-T scripts/mps2_m3.ld`	Uses the given linker script to describe memory and section placement
`-Xlinker -Map=build/output.map`	Makes the linker emit a map file of every placement and symbol address
`-nostdlib -lgcc`	Links no standard library, only `libgcc` for compiler support routines

- Target options must describe the actual machine: instruction set and Thumb execution, floating-point assumptions, freestanding environment, warning and optimisation policy. Host compiler defaults are not safe substitutes.
- Make variables keep the policy in one place:

Variable	Role
`CC`	Chooses the cross compiler (`arm-none-eabi-gcc`)
`CFLAGS`	Governs compilation of each source
`LDFLAGS`	Governs memory layout and the final link

> **DIAGNOSING A BUILD FAILURE**
>
> The full command line is the exact contract that produced the failure: copy it before changing anything.

- **Debug information** (`-g3`) is stored in the ELF for host tools: QEMU and a debugger can map an instruction address back to a C line, function, or variable. The target still executes only machine instructions; debug information does not change the boot image's behaviour.

1.4 Object Files, Symbols, and the Linker

An object file contains machine-code bytes, section names, symbol definitions, and **references whose final addresses are not yet known**. Each object is built independently, so its sections have no final position in Flash or SRAM.

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding \
    -c boot/startup.c -o build/boot/startup.o
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding \
    -c src/main.c -o build/src/main.o
arm-none-eabi-nm build/boot/startup.o build/src/main.o
```

`nm` output	Meaning
`U main` in `startup.o`	Undefined: referenced (by `Reset_Handler`) but not defined in this object

`nm` output	Meaning
`T main` in `main.o`	Defined, in the `.text` section

- Unresolved references are normal at this stage; neither object is a runnable program.
- The **linker**:
  - Concatenates matching sections from all objects.
  - Resolves every symbol (e.g., `main`, `_estack`) to a final address and patches all references.
  - Checks that every referenced symbol has exactly one valid definition.

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding \
    -T scripts/mps2_m3.ld -Xlinker -Map=build/output.map -nostdlib -lgcc \
    build/boot/startup.o build/src/main.o -o build/scratch-os.axf
arm-none-eabi-nm build/scratch-os.axf | grep -w main
```

- In the final image `main` is `T`: the `U` in `startup.o` has been connected to the `T` in `main.o`. The map file shows which object supplied `main` and its final address.
- The **linker script** (`.ld`) states where things go: physical addresses of `FLASH` and `RAM`, the contents of `.text`, `.data`, `.bss`, where the stack starts, and where the vector table must sit.
- On a desktop the default linker script is invisible; on bare metal it must be supplied, because only the developer knows the chip's memory map.
- Separate compilation followed by linking enables fast incremental builds.

2 ELF Image Format and Memory Layout

2.1 Sections, Segments, and the ELF Header

- ELF (Executable and Linkable Format) is the format of the final image `build/scratch-os.axf`; the `.axf` suffix conventionally denotes an **ARM executable** with debug information.
- The image is a 32-bit, little-endian ARM ELF executable with several views of the same bytes:
  - **ELF header** at offset 0: points to the section and program-header tables and names the entry point.
  - **Sections** (for the linker and debugger): `.text`, `.rodata`, `.data`, `.bss`, symbol tables, debug information.
  - **Program headers / segments** (for a loader): which chunks map to which memory at run time.
  - **Symbols**: names for final addresses.
- Inspection tools: `arm-none-eabi-readelf`, `arm-none-eabi-objdump`, `arm-none-eabi-nm`, `arm-none-eabi-size`.

Section	Contents	Placement
`.isr_vector`	Initial **stack pointer** and handler addresses	Start of Flash
`.text`	**Executable** instructions	Flash
`.rodata`	**Read-only** constants	Flash
`.data`	Writable **variables** with initial values	Runs in SRAM, initial image stored in Flash
`.bss`	Writable **zero-initialised** variables	SRAM

`arm-none-eabi-objdump -h` lists each section with two addresses:

- **VMA** (virtual memory address): the run address.
- **LMA** (load memory address): where the bytes are stored in the image.
- They differ only for `.data`: VMA in RAM, LMA in Flash. This difference is what the startup copy loop resolves.

2.2 Entry Point and the Thumb Bit

2.2.1 ARM and Thumb instruction sets

ARM cores define two instruction sets: **ARM** (32-bit instructions) and **Thumb** (16-bit instructions, extended to mixed 16/32-bit in Thumb-2). A core supporting both must know, at every branch, which set the target code uses.

> **IMPORTANT**
>
> The Cortex-M3 executes **only Thumb**.

Thumb instructions are 2 or 4 bytes long and always start at an **even** address. The linker places them in the `.text` output section, in Flash.

As a consequence, the bit 0 of the start address of an instruction is **always 0** and **carries no information**. ARM reuses it as a state flag in every **code pointer**, i.e. every stored address the CPU branches to (vector-table entry,

function pointer, ELF entry point, target of `bx` / `blx`):

- **Bit 0 = 1**: the target is **Thumb code**.
- **Bit 0 = 0**: the target is **ARM code**, which does not exist on Cortex-M and raises a **UsageFault** (invalid state).

2.2.2 How the linker produces `0x48` and `0x49`

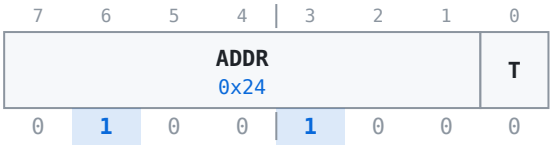
- **Placement**: the linker fills Flash in the order given by the linker script. The 64-byte vector table occupies `0x00` – `0x3F`; `.text` begins at `0x40` with the code of `startup.o`, inside which the first instruction of `Reset_Handler` lands at `0x48`. The symbol `Reset_Handler` therefore takes the value `0x48`.
- **Marking**: the compiler (or `.thumb_func` in assembly) records in the object file that `Reset_Handler` is a Thumb function.
- **Writing code pointers**: wherever the address of a Thumb function is stored as a branch target, the linker writes `address | 1` (bitwise OR: bit 0 set to 1, i.e. +1 for an even address):
  - Vector table word 1 (`.word Reset_Handler`): `0x48 | 1 = 0x49`.
  - Entry field of the ELF header (from `ENTRY(Reset_Handler)` in the linker script): `0x49`.
- **Reset**: the CPU loads `0x49` from vector word 1, checks bit 0 = 1 (Thumb, valid), sets the `T` bit of `xPSR`, clears bit 0, and starts executing with `PC = 0x48`.

```
0x48 = 0100 1000   location of the first instruction
0x49 = 0100 1001   code pointer: same location + Thumb flag in bit 0
```

The two values side by side: the address bits are identical, only bit 0 differs.

symbol 8-bit **= 0x48**

Low byte of Reset_Handler's address (map file, nm, objdump)



Bits	Field	Value	Description
7:1	ADDR	0x24 (36)	Instruction address bits
0	T	0 → even location	Bit 0 0 = even location, 1 = Thumb code pointer

pointer 8-bit **= 0x49**

Low byte of the code pointer (vector word 1, ELF entry)



Bits	Field	Value	Description
7:1	ADDR	0x24 (36)	Instruction address bits: the same instruction
0	T	1 → Thumb code pointer	Bit 0 0 = even location, 1 = Thumb code pointer

2.2.3 Inspection tools and what they report

Tool	What it is	Value for `Reset_Handler`
Map file `build/output.map`	Text report written by the linker during the link (`-Map=`): output sections, input pieces with their `.o` , address of every symbol	`0x00000048` (location)
`arm-none-eabi-nm`	Lists the symbols of an object or ELF file with address and type letter	`00000048 T Reset_Handler` (location)
`arm-none-eabi-objdump -d`	Disassembles the code: every instruction with the address where it is stored	`00000048 <Reset_Handler>:` (location)
`arm-none-eabi-readelf -h`	Prints the ELF header, including the entry point	`Entry point address: 0x49` (code pointer)

- Map file, `nm` , and `objdump` answer "where are the bytes stored?": even value.
- ELF entry point and vector table answer "how does the CPU branch there?": odd value.
- No instruction starts at `0x49` : the byte at `0x49` belongs to the instruction that starts at `0x48` . Both values identify the same instruction, once as a location and once as a branch target.
- Data pointers are unaffected: bit 0 has this meaning only for addresses used as branch targets (e.g., `&counter` is a plain address).

THUMB ADDRESS RULE

- Symbol address (where the bytes are stored): always even, e.g. `0x48` .
- Code-pointer value (how the CPU branches there): symbol address `| 1` , e.g. `0x49` .
- On Cortex-M, a code pointer with bit 0 = 0 raises a UsageFault.

2.3 Worked Layout from the Map File

2.3.1 What the map file is

The **map file** (`build/output.map` , also called *link map*) is a text report written by the linker while it builds the image, requested with `-Xlinker -Map=build/output.map` .

- It records every placement decision of the linker: where each piece of the program ended up in memory, how large it is, and which object file it came from.
- It is the reference for the question **"where did this function, variable, or constant end up?**".

```
Memory Configuration

Name             Origin             Length
FLASH            0x00000000         0x00400000
RAM              0x20000000         0x00400000

Linker script and memory map

.isr_vector     0x00000000       0x40
 .isr_vector    0x00000000       0x40 build/boot/startup.o

.text           0x00000040       0xa0
 .text          0x00000040       0x64 build/boot/startup.o
                0x00000048                Reset_Handler
 .text          0x000000a4       0x2c build/src/main.o
                0x000000bc                main
 .rodata        0x000000d0        0xe build/src/main.o
```

Line type	Example	Meaning
Memory Configuration row	`FLASH 0x00000000 0x00400000`	A `MEMORY` region of the linker script: name, start, length
Non-indented section line	`.text 0x00000040 0xa0`	An **output section** of the final image: start address and size
Indented line with a `.o`	`.text 0x00000040 0x64 build/boot/startup.o`	An **input piece** of that output section: address, size, source object file

Line type	Example	Meaning
Address followed by a name	`0x00000048 Reset_Handler`	A **symbol** (function or variable) and its final address

- The excerpt above is simplified; the real file also lists debug sections, discarded sections, and linker-script symbols.
- Static (file-local) functions and variables do not appear as symbols in the map file.

2.3.2 Placement rules that produce the layout

The linker fills each region following three ordering rules:

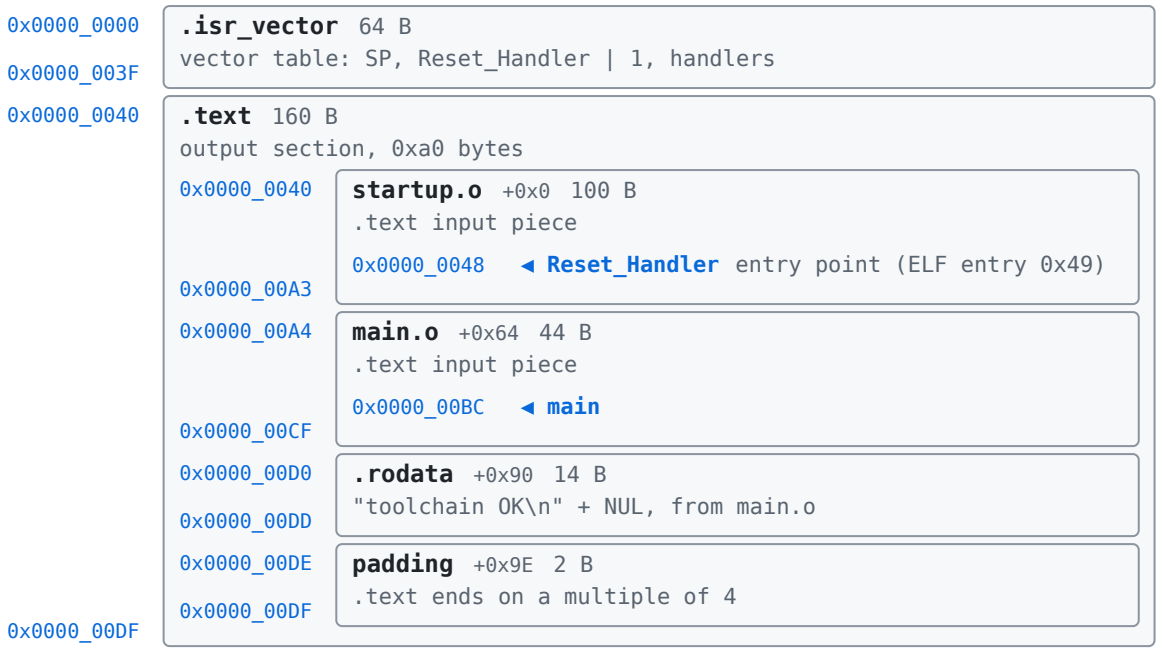
- **Order of output sections in the linker script**: `.isr_vector` first, then `.text`. The vector table (16 words × 4 bytes = 64 bytes = `0x40`) therefore starts at `0x00`, and `.text` starts right after it at `0x40`.
- **Order of object files on the link command line**: `startup.o` before `main.o`. The `0x64` bytes of `startup.o` occupy `0x40` – `0xA3`; the `0x2c` bytes of `main.o` follow at `0xA4` (`0x40 + 0x64`).
- **Order of rules inside `.text`**: `*(.text*)` (all code) before `*(.rodata*)` (all constants). The string literal therefore comes after **all** code, at `0xD0` (`0xA4 + 0x2C`), not next to `main`.

2.3.3 Flash Layout (`0x00` – `0xDF`, 224 bytes)

Address range	Size (bytes)	Content	Output section	From
`0x00` – `0x03`	4	Vector word 0: initial SP = `0x20400000`	`.isr_vector`	`startup.o`
`0x04` – `0x07`	4	Vector word 1: `Reset_Handler | 1` = `0x49`	`.isr_vector`	`startup.o`
`0x08` – `0x3F`	56	Vector words 2–15: addresses of the other exception handlers	`.isr_vector`	`startup.o`
`0x40` – `0x47`	8	Code of `startup.o` placed before `Reset_Handler`	`.text`	`startup.o`
`0x48` – `0xA3`	92	`Reset_Handler` (first instruction at `0x48`, the entry point) and the rest of the code of `startup.o`	`.text`	`startup.o`
`0xA4` – `0xBB`	24	Code of `main.o` placed before `main` (static helpers such as `semihost_write0`)	`.text`	`main.o`
`0xBC` – `0xCF`	20	`main`	`.text`	`main.o`
`0xD0` – `0xDD`	14	String literal `"toolchain OK\n"` plus terminating NUL	`.rodata`, folded into `.text`	`main.o`
`0xDE` – `0xDF`	2	Alignment padding (`.text` ends on a multiple of 4)	`.text`	linker

The same layout as a map, with the input pieces drawn inside their output section:

Figure 2.2 — Flash after linking (from build/output.map)

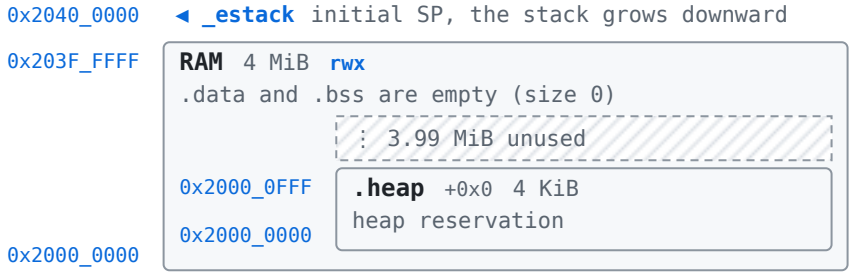


- End of `.text` : `0x40 + 0xA0 = 0xE0` , i.e. 224 bytes of Flash, the `text` value reported by `arm-none-eabi-size` .

2.3.4 RAM Layout (`0x20000000` – `0x203FFFFF` , 4 MiB)

Address range	Size (bytes)	Content	Output section
`0x20000000`	0	Initialised globals (none in this program)	`.data`
`0x20000000`	0	Zero-initialised globals (none in this program)	`.bss`
`0x20000000` – `0x20000FFF`	4096 (`0x1000`)	Heap reservation	`.heap`
`0x20001000` –...	—	Free RAM	—
...– `0x203FFFFF`	—	Stack, growing downward from `_estack` = `0x20400000`	—

Figure 2.3 — RAM after linking



- `.data` and `.bss` have size 0, so they occupy no bytes and `.heap` starts at the same address.

> **EMPTY `.data` AND `.bss`**
>
> - Size 0 only means that this program has no initialised or zero-initialised global variables.
> - The startup code must still copy `.data` and zero `.bss` in general: any global or static variable makes the same loops do visible work.

2.4 Flash and RAM Budgets

- `.bss` has a size in memory but usually no bytes in the ELF file: startup code creates its zero values.
- `.data` consumes **both** Flash (initial bytes) and SRAM (run-time copy).

```
Flash usage = code + constants + .data initial image
RAM usage   = .data + .bss + stack + heap reservation
```

`arm-none-eabi-size build/scratch-os.axf` reports `text` = 224, `data` = 0, `bss` = 4096 bytes:

- `text` = `0x40 + 0xa0` = 224 bytes (vector table plus code and constants).
- `bss` includes the 4096-byte heap reservation.
- `text + data` must fit in Flash; `data + bss` must fit in RAM: the same bytes of `.data` are counted against two budgets.

> **FILE SIZE IS NOT RAM USAGE**
>
> The ELF file size includes debug information and excludes `.bss`; memory budgets are read from the `size` columns and the link map.

3 Bare-Metal Runtime: Startup Code, Semihosting, and QEMU Execution

3.1 Startup Code Responsibilities

On a desktop, the OS loader and C runtime prepare memory and invoke the program. On a bare-metal target no such service exists at boot: the application supplies its own runtime.

- With `-nostartfiles`, the toolchain's startup objects are excluded and `boot/startup.c` takes over. The startup code must:
  - Provide the vector table.
  - Initialise `.data` (copy initial values from Flash to RAM) and `.bss` (fill with zeros).
  - Call constructors if the language or runtime requires them.
  - Call `main()` and define what happens if it returns.

After these steps, every global variable has its promised value when `main()` starts.

- `Default_Handler`: an **infinite loop** installed for unimplemented exception vectors. Stopping in a known loop is safer and more diagnosable than executing arbitrary memory after an unexpected exception.
- `main()` has **no operating system to return to**: if it returns, `Reset_Handler` enters a final `for( ;; ) {}` spin loop.

> ⚠ **QEMU DOES NOT EXIT**
>
> After `main()` returns, the target keeps executing the final spin loop, so QEMU keeps running and must be interrupted manually. The printed line is evidence that execution reached the intended point, not an exit notification.

3.2 Semihosting Output via `bkpt 0xab`

3.2.1 The problem

A bare-metal program has no console and no OS: `printf` needs a C library and an OS underneath (none present), and output through a UART needs a peripheral driver (not written yet).

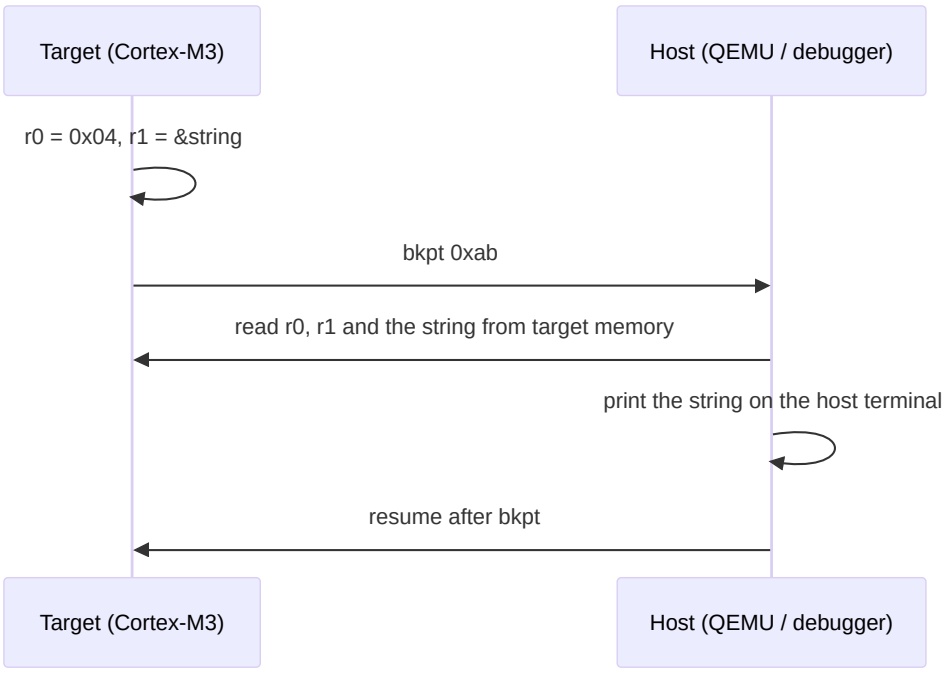
- **Semihosting** solves this during development: the program running on the target asks the **host** (the computer running QEMU or a debugger) to perform an I/O operation on its behalf. The target borrows the host's console, files, and clock.

3.2.2 How a Semihosting Request Works

A semihosting request follows a fixed convention, defined by ARM's semihosting specification and shared by the program and by every debugger or emulator that supports semihosting. The convention states where the program places its request and which instruction announces it. Because both sides follow the same rules, the host knows exactly where to look for the request and how to interpret it.

Element	Role in the request	Value for string output
`r0`	Operation number: which service is requested	`0x04` (`SYS_WRITE0` , "print a NUL-terminated string")
`r1`	Argument: the data the service needs	Address of the string in target memory
`bkpt 0xab`	Signal: the request is ready	—

Figure 2.4 — A semihosting request



On the target side, the program loads `0x04` into `r0`, loads the address of the string into `r1`, and executes `bkpt 0xab`. On the host side, QEMU notices the breakpoint and recognises the immediate value `0xab` as a semihosting request. It reads `r0` to learn which service is requested and `r1` to find the string, copies the bytes from the target's memory up to the terminating NUL, and prints them on the host terminal. The target then resumes at the instruction following `bkpt`, as if that instruction had done nothing.

The choice of a breakpoint instruction is deliberate. `bkpt` normally halts the processor and hands control to an attached debugger. The semihosting convention reuses this behaviour, and the immediate value `0xab` tells the debugger or emulator that the halt is a service request, not a breakpoint set by a developer.

This pattern is a **trap**: the program suspends its own work, control passes to a more powerful entity outside the CPU that performs the operation, and control then returns to the program. A system call in an operating system has the same shape, with the kernel playing the role of the host.

> ⚠ **DEVELOPMENT TOOL ONLY**
>
> Semihosting works only when a debugger, or an emulator started with semihosting enabled (`-semihosting` in QEMU), is attached and listening. On a real board with nothing attached, no one answers the request and `bkpt` simply halts the CPU. For this reason, production firmware produces its output through a peripheral, such as a UART driver, instead of semihosting.

3.2.3 Implementation in C

```c
#include <stdint.h>

static void semihost_write0( const char *message )
{
    register uint32_t     r0 __asm__( "r0" ) = 0x04;     /* SYS_WRITE0 */
    register const char *r1 __asm__( "r1" ) = message; /* string address */
    __asm__ volatile ( "bkpt 0xab" : : "r"( r0 ), "r"( r1 ) : "memory" );
}

int main( void )
{
    semihost_write0( "toolchain OK\n" );
    return 0;
}
```

Code	Meaning
`register uint32_t r0 __asm__( "r0" ) = 0x04;`	GCC extension: the variable is forced into register `r0` and holds the operation number `SYS_WRITE0`; standard C cannot choose registers
`register const char *r1 __asm__( "r1" ) = message;`	The string address is forced into register `r1`
`__asm__ volatile ( "bkpt 0xab" ... )`	Inline assembly (an assembly instruction embedded in C): inserts `bkpt 0xab`, which no C statement can express; `volatile` prevents the compiler from removing it
`: : "r"( r0 ), "r"( r1 )`	Input operands: guarantee that `r0` and `r1` hold their values when `bkpt` executes
`: "memory"`	Clobber: the instruction reads memory (the string), so all pending writes to memory are completed before it
`semihost_write0( "toolchain OK\n" );`	Prints the message on the host terminal: at C level the function means "the host prints `message`"

- `printf` is not used: it requires the C-library initialisation (newlib's semihosting-backed `stdio`) that the custom reset path does not perform.
- `arm-none-eabi-objdump -d build/scratch-os.axf | grep -A2 bkpt` shows the `bkpt 0xab` instruction inside the compiled image.

3.3 QEMU Board Model

```
qemu-system-arm -machine mps2-an385 -kernel build/scratch-os.axf -nographic -semihosting
```

- `qemu-system-arm` emulates an ARM system and selected board peripherals.
- `-machine mps2-an385` : the machine choice determines CPU, memory map, and available devices (here a Cortex-M3 board); it must match the memory model in the linker script.
- `-kernel` : the image to boot, which remains an ARM ELF, not a host (Linux or macOS) program.
- `-nographic` : target console routed to the host terminal.
- `-semihosting` : enables interception of `bkpt 0xab` requests.

3.4 What the Output Proves and Failure Localisation

Seeing `toolchain OK` proves that several independent contracts agree:

- The cross compiler emitted valid target instructions.
- The linker placed startup and application code at usable addresses.
- The processor found the vector table and reset reached `Reset_Handler`.
- Startup code established the runtime state and reached `main()`.
- The output mechanism (semihosting) was enabled and recognised by QEMU.

It does not prove that future drivers or timing properties are correct: it is a narrow but valuable check.

When QEMU prints nothing, work outward from the earliest guaranteed event:

- Confirm the ELF was rebuilt and passed to QEMU.
- Inspect vector-table and section addresses.
- Check that the machine name matches the linker script's memory model.
- Confirm startup reaches `main()` .
- Check the output configuration (semihosting options, call to `semihost_write0`, console routing).

Symptom	Where to look
Compiler rejects a command	Cross-toolchain installation and Makefile flags
Unresolved symbol	Object list and matching declarations
Section overflow	Linker-script memory regions and the map file
QEMU starts but prints nothing	Reset path, output call, semihosting options

- Disciplined build loop: build with `make`, read the command and the **first** diagnostic, run the ELF in QEMU, compare observed and expected output.
- Compilation failures, boot failures, and program-logic bugs must be kept separate.

> **LOCATE THE FAILING BOUNDARY**
>
> Changing random compiler flags is not a diagnosis; the first step is to identify which boundary of the pipeline failed.

4 Build Automation with Make

4.1 Rules, Prerequisites, and Timestamp-Based Rebuilds

`make` is not a compiler: it reads a `Makefile` describing a graph of **targets** and runs only the commands needed to bring the requested targets up to date. The alternative, typing every compiler command by hand, is slow and error-prone; `make` is the reproducible build.

```
target: prerequisite1 prerequisite2
          recipe line 1
          recipe line 2
```

- `target` : normally a file to produce (object file, ELF image).
- `prerequisites` : files that must be current before the recipe runs.
- `recipe` : shell commands that produce the target; **every recipe line starts with a literal Tab**, not spaces.
- A target is **out of date** and its recipe runs if the target does not exist, or if any prerequisite is newer (by modification time) than the target.
- Targets may depend on other targets, forming a **dependency graph** walked bottom-up:

```
main.c changed      --> main.o rebuilt --> scratch-os.axf relinked
startup.c unchanged --> startup.o reused
```

- The rebuild logic is correct only if the graph is complete:
  - Header dependencies must be tracked so that an interface change recompiles the affected sources.
  - A linker script that is not listed as a prerequisite does not trigger a relink when it changes; `make clean && make` forces a full rebuild.

4.2 Diagnosing the Missing Separator Error

```
CC := arm-none-eabi-gcc
BUILD_DIR := build

$(BUILD_DIR)/%.o: %.c
    @mkdir -p $(dir $@)
         $(CC) $(CFLAGS) -c $< -o $@
```

```
$ make
Makefile:5: *** missing separator.  Stop.
```

- `Makefile:5` is exact: line 5 is indented with four spaces instead of a Tab.
- `missing separator` describes what the parser expected (the Tab that marks a recipe line), not what is written: without it, `make` tried to parse the line as an assignment or a rule and failed.
- Spaces and a Tab look identical in an editor; the defect is visible only by rendering the bytes:

```
$ cat -et Makefile
CC := arm-none-eabi-gcc$
BUILD_DIR := build$
$
$(BUILD_DIR)/%.o: %.c$
    @mkdir -p $(dir $@)$
^I$(CC) $(CFLAGS) -c $< -o $@$
```

- `cat -et` marks each line end with `$` and each Tab with `^I`; line 5 lacks the `^I`.
- Portability:
  - `cat -A` exists only in GNU `cat`; BSD `cat` rejects it. `cat -et` works on both.
  - `sed -n l` is a portable alternative (Tab shown as `\t`), but BSD `sed` wraps long lines at about 70 columns.
- General technique: when a tool rejects a line that looks correct, render its bytes instead of rereading its characters.
- Editors can be configured not to expand Tabs in Makefiles.

4.3 Variables and Assignment Flavours

```
CC := arm-none-eabi-gcc
CFLAGS := -mthumb -mcpu=cortex-m3
CFLAGS += -Wall -Wextra
```

Operator	Name	Behaviour
`:=`	Simply expanded	Right-hand side evaluated once, immediately (preferred default)
`=`	Recursively expanded	Re-evaluated at every use; can be slow or loop if self-referential
`+=`	Append	Adds to the current value
`?=`	Conditional	Assigns only if not already set; allows overrides such as `make CC=clang` or the environment

- Variables are expanded with `$(NAME)`.
- Convention: uppercase names (`CC`, `CFLAGS`, `LDFLAGS`) are the tool and flags used by recipes; `make`'s built-in rules look for these exact names. Other names (`SOURCES`, `OBJECTS`, `BUILD_DIR`) hold the Makefile's own bookkeeping.

4.4 Automatic Variables, Pattern Rules, and Wildcards

Variable	Meaning
`$@`	Target of the rule

Variable	Meaning
`$<`	First prerequisite
`$^`	All prerequisites, space-separated, duplicates removed
`$*`	Stem matched by `%` in a pattern rule
`$(dir $@)`	Directory part of `$@` (a built-in function, not an automatic variable)

```
$(BUILD_DIR)/%.o: %.c
	@mkdir -p $(dir $@)
	$(CC) $(CFLAGS) -c $< -o $@
```

- `%.o: %.c` is a **pattern rule**: `%` is a stem that must match on both sides. For `build/src/main.o`, the stem is `src/main` and the prerequisite is `src/main.c`. One rule covers every object file.
- `$(dir $@)` (e.g., `build/boot/`) lets `mkdir -p` create the output directory before the compiler writes into it; `$<` is the `.c` file; `$@` is the `.o` being produced.
- `%` is a `make` mechanism, distinct from shell globbing. The glob equivalent is the `$(wildcard ...)` function, evaluated when the Makefile is read:

```
SOURCES := $(wildcard boot/*.c src/*.c)
```

Source list style	Advantage	Risk
Explicit (`SOURCES := boot/startup.c src/main.c`)	Easy to audit; a renamed file fails loudly	Must be maintained by hand
`$(wildcard ...)`	New files picked up automatically	Silently includes or excludes files

- An explicit list suits small trees; `$(wildcard ...)` suits large trees where manual maintenance is the bigger risk.
- `$(SOURCES:%.c=$(BUILD_DIR)/%.o)` is a **substitution reference**, shorthand for `$(patsubst %.c,$(BUILD_DIR)/%.o,$(SOURCES))`. It maps the source list to `build/boot/startup.o build/src/main.o`, connecting it to the pattern rule.

4.5 Phony Targets and the Default Goal

```
.PHONY: all clean run

all: $(BIN)

run: $(BIN)
	qemu-system-arm -machine mps2-an385 ...

clean:
```

```
	rm -rf $(BUILD_DIR)
```

- `run` and `clean` name actions, not files. If a file named `clean` existed, `make` would consider the target up to date and skip the recipe. `.PHONY` declares that a target is not a file, so its recipe always runs.
- `all` is the conventional **default goal**: the first target in the file, built when `make` is invoked without arguments. With `$(BIN)` as its only prerequisite it is an alias for the real image.

4.6 Inspecting Makefile State

A Makefile can be wrong without failing (e.g., a wrong substitution silently builds the wrong list). The useful information is what variables *expanded to*, not what the text says.

```
SOURCES := boot/startup.c src/main.c
BUILD_DIR := build
OBJECTS := $(SOURCES:%.c=$(BUILD_DIR)/%.o)
CFLAGS := -mthumb -mcpu=cortex-m3
CFLAGS += -Wall -Wextra

$(info OBJECTS = $(OBJECTS))
$(warning CFLAGS is $(CFLAGS))

print-%:
	@echo "$* = $($*)"

all:
	@echo linking
```

```
$ make all
OBJECTS = build/boot/startup.o build/src/main.o
Makefile:8: CFLAGS is -mthumb -mcpu=cortex-m3 -Wall -Wextra
linking
```

- `$(info ...)` and `$(warning ...)` are expanded while the Makefile is **read**, before any recipe runs: they show the value at that point in the file (after the `+=` here).
- `$(warning ...)` adds the `file:line:` prefix, useful when several are present.
- Both fire on every invocation, including `make clean`: they are temporary debugging aids.
- `print-%` uses the pattern mechanism and `$*` to query any variable: `make print-CFLAGS` prints `CFLAGS = -mthumb -mcpu=cortex-m3 -Wall -Wextra`.
- `make -n` prints the commands a goal would run without executing them, to verify that `$<` and `$@` expand as intended.
- `make -p` dumps the whole database (over 1300 lines for a 14-line Makefile, mostly built-in defaults); useful only when the cause lies in something not written in the Makefile.

4.7 The Complete Build Graph

```
SOURCES := boot/startup.c src/main.c
OBJECTS := $(SOURCES:%.c=$(BUILD_DIR)/%.o)

.PHONY: all clean run
all: $(BIN)

$(BUILD_DIR)/%.o: %.c
        @mkdir -p $(dir $@)
        $(CC) $(CFLAGS) -c $< -o $@

$(BIN): $(OBJECTS)
        $(CC) $(CFLAGS) $(LDFLAGS) $(OBJECTS) -o $(BIN)

run: $(BIN)
        qemu-system-arm ...

clean:
        rm -rf $(BUILD_DIR)
```

- Three-level graph: `run` depends on `$(BIN)`, `$(BIN)` on `$(OBJECTS)`, each object on its `.c` file.
- `make` (equivalent to `make all`): builds missing or stale objects through the pattern rule, then runs the link recipe. Running it again with no changes does nothing.
- `make run`: rebuilds anything stale, then boots the image in QEMU.
- The graph is the record of what the image is made of; re-running it is correct whether nothing changed, one file changed, or `build/` does not exist.
- A separate **build directory** holds generated objects, dependency files, maps, and ELF images:
  - Source directories stay readable.
  - `clean` can remove all generated files safely.
  - Different configurations can coexist.
  - A fresh build tests whether the dependency graph is complete.
- Generated objects and ELF bytes are never edited by hand to change program behaviour.